INTEGRAL DRAGON

MODULE 3 • SESSION 10

Task Decomposition and Subagents

Break the work down before you delegate it

AI-Accelerated Software Development | Foundations Program

Confidential. Prepared exclusively for Ensemble Health Partners.

Giant prompts fail for structural reasons — not because the model isn't capable enough

WHY THEY FAIL

- Too many goals mixed together in one request
- Unclear boundaries between what counts as done
- Noisy intermediate output floods the working context
- No explicit contract for what should come back
- Mixed concerns make the output hard to review or trust

These are structural failures — they exist before the model runs. A better model does not fix bad task design.

WHAT SUBAGENTS PROVIDE

- Fresh context — no accumulated noise from the main session
- Focused role — one job, one outcome
- Optional tool restrictions — tighter capability scope
- Concise return path — summary only, not the full trace

Session focus: The real skill is not delegation syntax. It is task decomposition — defining bounded units of work that can be delegated cleanly.

Good delegated tasks are narrow and explicit — bad ones sound like mini-projects

Well-formed delegated tasks

```
› "Review the auth module for security issues
    and return a ranked summary of findings."

› "Trace the failing tests and return only
    the root causes — not the full trace."

› "Map files touched by this feature and
    list the likely risk areas."

› "Extract duplicate logic across these
    3 files and propose a single abstraction."
```

Each has one objective, one scope boundary, and one clear return deliverable.

Poorly-formed delegated tasks

```
× "Build the whole feature."

× "Fix everything wrong with this repo."

× "Refactor the app and improve security."

× "Help me with the billing dashboard."
```

These are not task boundaries. They are wishful thinking delegated to a machine. The model will attempt them — and produce plausible-looking, hard-to-review output.

Every delegated task needs a contract — five things that must be defined before the subagent runs

1. OBJECTIVE

One concrete goal. A single sentence describing what success looks like. If you need two sentences, you have two tasks.

2. SCOPE BOUNDARY

Exactly which files, modules, systems, or data are in scope. Anything not listed is out of scope — explicitly. The model should not guess at the edges.

3. ALLOWED TOOLS

Which tools the subagent may use. Read-only for reviewers. Write access only for implementers. Tighter permissions reduce blast radius when the scope is unclear.

4. EXPECTED DELIVERABLE

The exact format of what comes back. A ranked list? A diff? A one-paragraph summary? A JSON object? The main session must be able to consume the output without interpretation.

5. STOP CONDITION

When is the task done? What should the subagent do if it hits a blocker? An undefined stop condition leads to scope creep inside the delegated session — the subagent keeps going because nothing told it to stop.

Use subagents when isolation helps — stay in the main conversation when continuity matters

Use a subagent when...

Output would pollute the main context

Verbose test traces, large search results, noisy research output — these do not belong in the main conversation. The subagent compresses them into a summary before returning.

The work is genuinely self-contained

A bounded review, an isolated analysis, a specialist task that has a single clean input and a single useful output. No back-and-forth needed during execution.

A tighter permission boundary is valuable

When the task should only read, not write. When only specific tools should be available. A subagent can operate with stricter tool access than the main session.

Stay in the main conversation when...

Rapid refinement is needed

Back-and-forth clarification, in-flight adjustments, quick pivots based on partial output. Subagent startup overhead would dominate the value.

Planning and implementation are tightly coupled

When the plan changes the implementation and the implementation changes the plan in real time. Isolation would sever the feedback loop that makes the work coherent.

The task is small and targeted

One-line bugfixes, quick edits, short targeted changes. The cost of context isolation exceeds the benefit when the task takes thirty seconds either way.

LIVE DEMO — SESSION 10

One focused reviewer subagent. One bounded area. One summary.

Define the contract. Delegate the work. Bring back only what matters.

Bounded tasks improve accuracy and reduce cost — not because of the model, but because of the structure

Demo sequence

```
# Step 1: Define the reviewer subagent
Role: read-only code reviewer
Scope: auth/ directory only
Tools: read_file, search
Deliverable: ranked findings, max 5

# Step 2: Delegate
> "Review auth/ for security issues.
    Return: issue, severity, location.
    Stop after 5 findings."

# Step 3: Receive the summary
→ 5-item ranked list returned
→ No trace noise in main context
→ Human reviews and decides next step
```

Why this improves outcomes

Less irrelevant context

The subagent's working memory is clean. It sees only the auth module, not the last 40 messages of the main session. Signal-to-noise ratio is higher from the start.

Fewer mixed objectives

It has one job. It cannot accidentally drift into implementation while trying to review. Bounded scope prevents the model from trying to help beyond what was asked.

Clearer human review point

A five-item ranked list is reviewable in thirty seconds. The human decides what to act on. The engineer remains the decision-maker; the subagent is the research instrument.

The most common decomposition failures happen before the subagent runs — they are design errors, not execution errors

Delegating before defining inputs and outputs

Sending a subagent without specifying what format to return, or what data to start from. The model will invent a structure — which may be completely different from what the main session expects.

Giving broad permissions to a narrow task

A read-only reviewer does not need write access. An analysis task does not need the ability to execute shell commands. Overly broad permissions expand the blast radius when the task drifts.

Using a subagent for work that needs constant interaction

Trying to delegate back-and-forth planning, live decision-making, or tasks that require multiple clarification rounds. The subagent completes in isolation and can't ask for help mid-run.

Asking for a summary with no criteria

"Summarize your findings" with no definition of what matters, what format to use, or how many items to return. The result is unpredictable and hard to use as input to the next step.

Root cause: All four failures are upstream. They occur when the engineer delegates before designing the task. Decomposition quality is an engineering responsibility — not a model feature.

SESSION 10 — KEY TAKEAWAY

Subagents work best when the task is atomic

Atomic task design is an engineering responsibility — not a model feature.

Next: Session 11 — Session 10 is hierarchical delegation. Session 11 expands that into coordinated parallel teams with shared state and new failure modes.